

ASSIGNMENT REPORT

Module: ADVANCED ALGORITHMS

The 0/1 Knapsack Problem

Formulation, Heuristics, and Metaheuristics

Prepared by:

Oualid AMTOUT

Academic Program:

ASEDS (Advanced Software Engineering for Digital Services)

Project Repository:

<https://github.com/amtout-oualid/knapsack-problem.git>

Contents

1	Description of the Knapsack Problem	1
2	Mathematical Formulation	1
3	Exact Solution using IBM ILOG CPLEX	1
4	Implementation of 5 Greedy Heuristics	3
4.1	Maximum Utility Greedy Heuristic	3
4.2	Minimum Weight Greedy Heuristic	3
4.3	Maximum Ratio Greedy Heuristic (Density)	4
4.4	Decreasing-Weight Based Greedy (Tie-Breaking)	4
4.5	Randomized Greedy Heuristic	5
5	Variable Neighborhood Descent (VND)	6
5.1	Neighborhood Structures: 1-Flip and Swap	6
5.2	Sequential VND Algorithm	6
6	Variable Neighborhood Search (VNS)	7
6.1	VNS Components Description	7
7	Comparative Analysis of Results	8
7.1	Performance Summary	8
7.2	Discussion of Findings	8
8	Generalization to the Multidimensional Knapsack Problem (MKP)	9
8.1	Mathematical Formulation	9
8.2	Algorithm Adaptations	10
A	Python Source Code	11
A.1	Final Discussion	11

List of Figures

1	execution results for VNS.	8
2	Combined execution results for VND and VNS.	9

List of Tables

1	Comprehensive performance comparison across all methods.	9
---	--	---

1 Description of the Knapsack Problem

The 0/1 Knapsack problem is a fundamental combinatorial optimization problem in the field of computer science and operations research. The general premise involves filling a fixed-capacity “knapsack” (or container) with a subset of items selected from a given list, where each item has associated weights and utilities, such that the total utility is maximized while not exceeding the knapsack’s weight constraint. The “0/1” constraint specifically denotes that an entire item must be either included in or excluded from the knapsack.

2 Mathematical Formulation

The 0/1 Knapsack problem can be formulated as an Integer Linear Program (ILP).

Parameters:

- n : Total number of available items.
- u_i : Utility (or profit) of item i .
- w_i : Weight of item i .
- W : Maximum weight capacity of the knapsack.

Decision Variables: Let x_i be a binary variable where:

- $x_i = 1$ if item i is selected to be placed in the knapsack.
- $x_i = 0$ if item i is not selected.

Objective Function: Maximize the total utility of the selected items:

$$\text{Maximize } Z = \sum_{i=1}^n u_i x_i$$

Constraints: The total weight of the selected items cannot exceed the knapsack capacity:

$$\sum_{i=1}^n w_i x_i \leq W$$

Variable Domains:

$$x_i \in \{0, 1\}, \quad \forall i \in \{1, \dots, n\}$$

3 Exact Solution using IBM ILOG CPLEX

To find the exact optimal solution for the 0-1 Knapsack problem, the mathematical formulation is modeled directly using IBM ILOG CPLEX Optimization Studio via its native Optimization Programming Language (OPL). This exact solution will serve as the benchmark to calculate the optimality gap of our approximate algorithms.

Mathematical Logic: The problem is modeled by defining the decision variables $x_i \in \{0, 1\}$. The objective is set to maximize the sum of $u_i x_i$. A single linear constraint is added to ensure that the sum of $w_i x_i$ is strictly less than or equal to the knapsack capacity W . CPLEX then uses exact methods (Branch and Cut) to find the global optimum.

Model File (.mod)

```

1  int n = ...;
2  int W = ...;
3  range Items = 1..n;
4  int u[Items] = ...;
5  int w[Items] = ...;
6
7  dvar boolean x[Items];
8
9  maximize
10     sum(i in Items) u[i] * x[i];
11
12  subject to {
13     sum(i in Items) w[i] * x[i] <= W;
14  }

```

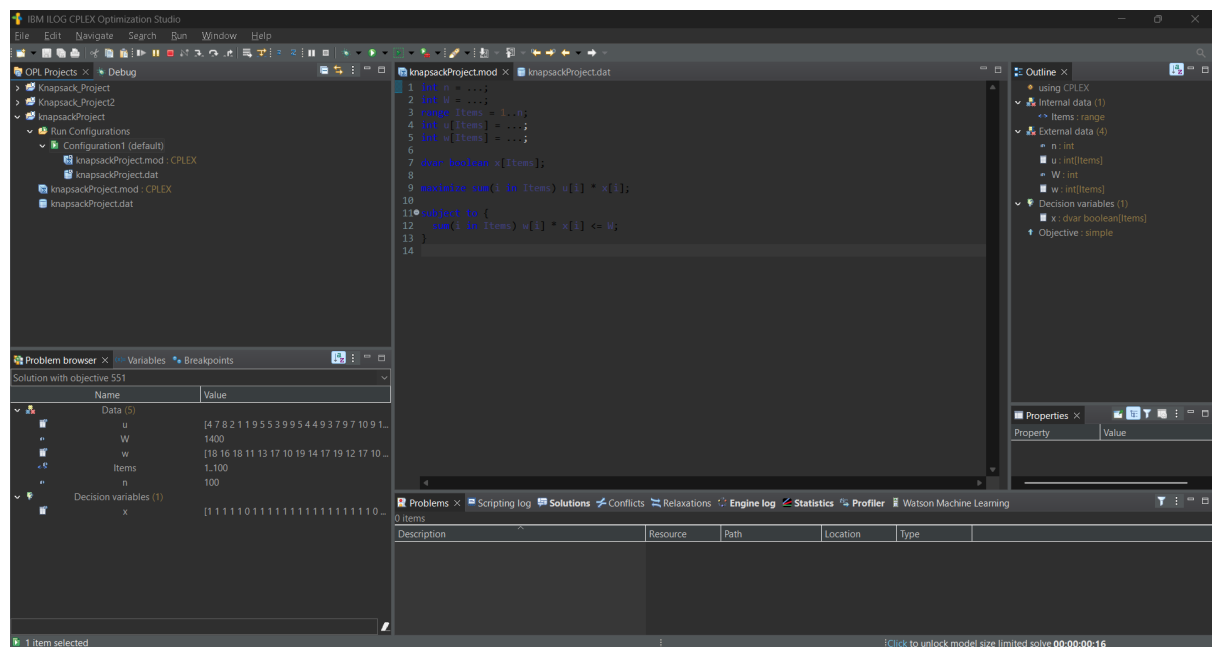
Data File (.dat)

```

1 n = 100;
2 W = 1400;
3
4 u = [4, 7, 8, 2, 1, 9, 5...];
5 w = [18, 16, 18, 11, 13...];
6
7 // (Full data arrays omitted
8 // for brevity)

```

Interpretation: For the first instance ($n = 100$), CPLEX successfully explores the solution space and returns the exact global optimum of $Z = 551$. This optimal value is crucial as it serves as the baseline for calculating the *Gap*(%) of our approximate methods. However, the license limitation encountered with the larger instance ($n = 2000$) perfectly illustrates why heuristic and metaheuristic approaches are practically necessary when exact solvers are constrained.



Note on Instance 2 ($n = 2000$):

The second instance could not be tested using this exact method. The Community Edition of IBM ILOG CPLEX is limited to a maximum of 1000 variables, making it impossible to run the model for $n = 2000$.

4 Implementation of 5 Greedy Heuristics

Greedy heuristics build a solution iteratively by making a locally optimal choice based on a specific sorting criterion. Below are 5 variations of the greedy approach.

4.1 Maximum Utility Greedy Heuristic

This approach sorts items based solely on their utility, aiming to pack the most valuable items first regardless of their weight.

Algorithm 1: Max Utility

1. Sort items descending by u_i
2. $C \leftarrow W$; $S \leftarrow \emptyset$
3. **for each** item i in sorted order:
4. **if** $w_i \leq C$ **then**
5. $S \leftarrow S \cup \{i\}$; $C \leftarrow C - w_i$
6. **return** S

```
1 def greedy_max_utility(n,W,u,w):
2     items = sorted(range(n), key=
3         lambda i: u[i], reverse=True)
4     sol, cur_w = [], 0
5     for i in items:
6         if cur_w + w[i] <= W:
7             sol.append(i)
8             cur_w += w[i]
9     return sol
```

```
PS C:\Users\pc\Desktop\knapsac> python .\Heuristics.py
--- 1. Max Utility Greedy ---
Utility (Z): 551 | Weight: 1398/1400 | CPU Time: 0.000249s

PS C:\Users\pc\Desktop\knapsac> █
```

Interpretation: Surprisingly, for this specific dataset, the Max Utility heuristic achieves the optimal value of 551. However, it nearly exhausts the capacity (1398/1400), indicating it greedily selected heavy, high-value items. While successful here, this approach generally risks leaving out multiple lighter items that could combine for a higher total utility in other instances.

4.2 Minimum Weight Greedy Heuristic

Prioritizes the lightest items first to pack the maximum quantity of items, assuming quantity yields higher total utility.

Algorithm 2: Min Weight

1. Sort items ascending by w_i
2. $C \leftarrow W$; $S \leftarrow \emptyset$
3. **for each** item i in sorted order:
4. **if** $w_i \leq C$ **then**
5. $S \leftarrow S \cup \{i\}$; $C \leftarrow C - w_i$
6. **return** S

```
1 def greedy_min_weight(n,W,u,w):
2     items = sorted(range(n), key=
3         lambda i: w[i])
4     sol, cur_w = [], 0
5     for i in items:
6         if cur_w + w[i] <= W:
7             sol.append(i)
8             cur_w += w[i]
9     return sol
```

Interpretation: This heuristic underperforms, yielding a utility of 532. By focusing purely on weight, it frequently selects items with extremely low profit values, proving that maximizing the quantity of packed items does not guarantee maximum total utility.

```
PS C:\Users\pc\Desktop\knapsac> python .\Heuristics.py
--- 2. Min Weight Greedy ---
Utility (Z): 532 | Weight: 1394/1400 | CPU Time: 0.000123s

PS C:\Users\pc\Desktop\knapsac> █
```

4.3 Maximum Ratio Greedy Heuristic (Density)

Evaluates the profit density by calculating the ratio of utility to weight ($r_i = u_i/w_i$). It prioritizes items with the highest density.

Algorithm 3: Max Ratio

1. Calculate $r_i = u_i/w_i$
2. Sort items descending by r_i
3. $C \leftarrow W$; $S \leftarrow \emptyset$
4. **for each** item i in sorted order:
5. **if** $w_i \leq C$ **then**
6. $S \leftarrow S \cup \{i\}$; $C \leftarrow C - w_i$
7. **return** S

```
1 def greedy_max_ratio(n,W,u,w):
2     items = sorted(range(n), key=
3         lambda i: u[i]/w[i], reverse=
4         True)
5     sol, cur_w = [], 0
6     for i in items:
7         if cur_w + w[i] <= W:
8             sol.append(i)
9             cur_w += w[i]
10    return sol
```

```
PS C:\Users\pc\Desktop\knapsac> python .\Heuristics.py
--- 3. Max Ratio (Density) Greedy ---
Utility (Z): 551 | Weight: 1397/1400 | CPU Time: 0.000190s

PS C:\Users\pc\Desktop\knapsac> █
```

Interpretation: This approach successfully finds the optimal solution ($Z = 551$) while using slightly less capacity (1397/1400) than the Max Utility method. It demonstrates that prioritizing profit density is generally the most robust and logical deterministic greedy strategy.

4.4 Decreasing-Weight Based Greedy (Tie-Breaking)

If two items have the same density ratio (r_i), they are ranked in decreasing order of weight to aggressively utilize capacity.

Algorithm 4: Tie-Breaking

1. Calculate $r_i = u_i/w_i$
2. Sort items descending by r_i
3. **If** equal r_i , sort desc by w_i
4. $C \leftarrow W$; $S \leftarrow \emptyset$
5. **for each** item i in sorted:
6. **if** $w_i \leq C$ **then**
7. $S \leftarrow S \cup \{i\}$; $C \leftarrow C - w_i$

```
1 def greedy_tie_break(n,W,u,w):
2     items = sorted(range(n), key=
3         lambda i: (u[i]/w[i], w[i]),
4         reverse=True)
5     sol, cur_w = [], 0
6     for i in items:
7         if cur_w + w[i] <= W:
8             sol.append(i)
9             cur_w += w[i]
10    return sol
```

```
PS C:\Users\pc\Desktop\knapsac> python .\Heuristics.py
--- 4. Tie-Breaking Greedy ---
Utility (Z): 551 | Weight: 1397/1400 | CPU Time: 0.000377s

PS C:\Users\pc\Desktop\knapsac>
```

Interpretation: The results are identical to the standard Max Ratio heuristic ($Z = 551$). This indicates that for this specific instance, either there were no significant ties in density ratios, or the tie-breaking mechanism selected the same items as Python’s default stable sort.

4.5 Randomized Greedy Heuristic

Introduces stochasticity by selecting an item randomly from a Restricted Candidate List (RCL) containing the top k items, preventing entrapment in deterministic local optima.

Algorithm 5: Randomized

1. Compute $r_i = u_i/w_i$
2. **while** available items fit:
3. Sort fitting items by r_i
4. Create RCL with top k items
5. Pick random i from RCL
6. $S \leftarrow S \cup \{i\}$; $C \leftarrow C - w_i$
7. **return** S

```
1 import random
2 def greedy_random(n,W,u,w,k=3):
3     items = sorted(range(n), key=
4         lambda i: u[i]/w[i], reverse=
5         True)
6     sol, cur_w = [], 0
7     avail = [i for i in items if cur_w
8         +w[i]<=W]
9     while avail:
10         ch = random.choice(avail[:k])
11         sol.append(ch)
12         cur_w += w[ch]
13         items.remove(ch)
14         avail = [i for i in items if
15             cur_w+w[i]<=W]
16     return sol
```

```
PS C:\Users\pc\Desktop\knapsac> python .\Heuristics.py
--- 5. Randomized Greedy ---
Utility (Z): 550 | Weight: 1394/1400 | CPU Time: 0.000643s

PS C:\Users\pc\Desktop\knapsac>
```

Interpretation: A single run of the randomized heuristic yields a slightly suboptimal value ($Z = 550$). However, its strength lies in multi-start frameworks; running it iteratively generates diverse, high-quality starting points that are crucial for initializing metaheuristics like VND and VNS.

5 Variable Neighborhood Descent (VND)

VND is a deterministic metaheuristic that escapes local optima by systematically switching between neighborhood structures. We utilize 1-Flip (N_1) and Swap (N_2) neighborhoods. The algorithm explores N_1 , and if an improvement is found, it restarts from N_1 . If not, it moves to N_2 .

5.1 Neighborhood Structures: 1-Flip and Swap

The local search algorithms iteratively explore the neighborhood to find a strictly better solution (First Improvement strategy).

Algorithms: Local Search (N_1 & N_2)

```
1. improved  $\leftarrow$  True
2. while improved do
3.   improved  $\leftarrow$  False
4.   for each move in Neighborhood do
5.      $x' \leftarrow$  Apply move (Flip or Swap)
6.     if  $x'$  is feasible &  $f(x') > f(x)$ :
7.        $x \leftarrow x'$ 
8.       improved  $\leftarrow$  True; break
9. return  $x$ 
```

```
1 def ls_1_flip(sol):
2     curr = list(sol)
3     for i in range(n):
4         curr[i] = 1 - curr[i] # Flip
5         item
6         if is_feasible(curr) and eval(
7             curr) > eval(sol):
8             return curr # First
9             improvement
10            curr[i] = 1 - curr[i] # Revert
11        return curr
12
13 def ls_swap(sol):
14     # Evaluates swapping 1 item in the
15     # knapsack with 1 item outside.
16     # Implementation omitted for
17     # brevity.
18     pass
```

5.2 Sequential VND Algorithm

The main VND loop alternates between N_1 and N_2 based on the success of the local search.

Algorithm 9: Sequential VND

```
1.  $k \leftarrow 1$ 
2. while  $k \leq k_{max}$  do
3.    $x' \leftarrow$  LocalSearch( $x, N_k$ )
4.   if  $x'$  feasible &  $f(x') > f(x)$ :
5.      $x \leftarrow x'$ 
6.      $k \leftarrow 1$ 
7.   else:
8.      $k \leftarrow k + 1$ 
9. return  $x$ 
```

```
1 def run_vnd(x_init):
2     x = list(x_init)
3     k, k_max = 1, 2
4
5     while k <= k_max:
6         if k == 1:
7             x_p = ls_1_flip(x)
8         else:
9             x_p = ls_swap(x)
10
11         if eval(x_p) > eval(x):
12             x, k = list(x_p), 1
13         else:
14             k += 1
15
16     return x
```

Interpretation: Starting from a sub-optimal initial greedy solution ($Z = 532$), the VND algorithm successfully enhances the solution quality, reaching $Z = 547$. By alternating between 1-Flip and Swap neighborhoods, it escapes the initial state but ultimately gets trapped in a stronger local optimum. It fails to reach the global optimum of 551, which perfectly highlights the limitations of purely deterministic local search methods and sets the stage for the necessity of a "shaking" mechanism.

```
PS C:\Users\pc\Desktop\knapsac> python .\VND.py
=====
METAHEURISTIC  | UTILITY (Z)  | CPU TIME
=====
VND             | 547          | 0.000663 s
VNS             | 551          | 2.000018 s
=====
PS C:\Users\pc\Desktop\knapsac> █
```

6 Variable Neighborhood Search (VNS)

VNS is a stochastic metaheuristic designed to escape local optima by alternating between intensification (local search) and diversification (shaking).

6.1 VNS Components Description

- **Neighborhood Structures (N_k):** We define two structures: N_1 (1-Flip), which changes the state of a single item, and N_2 (Swap), which exchanges an item inside the knapsack with one outside.
- **Shaking Function:** This function performs a random perturbation by flipping k random variables. This move is essential to diversify the search and "jump" to a completely different region of the solution space.
- **Local Search Procedure:** We use the previously defined **VND** as the intensification engine. After each shake, VND is called to find the best possible local optimum in the new region.

Algorithm 14: VNS

```
1. while stopping criterion not met:
2.    $k \leftarrow 1$ 
3.   while  $k \leq k_{max}$  do:
4.      $x' \leftarrow \text{Shaking}(x, N_k)$ 
5.      $x'' \leftarrow \text{VND}(x')$ 
6.     if  $f(x'') > f(x)$ :
7.        $x \leftarrow x''$ ;  $k \leftarrow 1$ 
8.     else:  $k \leftarrow k + 1$ 
9. return  $x$ 
```

```
1 def run_vns(x_init, n, W, u, w):
2     import time
3     x, start = x_init, time.time()
4     while time.time() - start < 2.0:
5         k, k_max = 1, 5
6         while k <= k_max:
7             x_p = shaking_1_flip(x, k)
8             x_pp = run_vnd(x_p, n, W, u, w
9         )
10            if eval(x_pp, u) > eval(x, u):
11                x, k = x_pp, 1
12            else:
13                k += 1
14    return x
```

```
PS C:\Users\pc\Desktop\knapsac> python .\VNS.py
-----
Variable Neighborhood Search (VNS) Execution
-----
[*] Final Utility (Z): 551
[*] CPU Time: 2.002828 seconds
-----
PS C:\Users\pc\Desktop\knapsac> █
```

Figure 1: execution results for VNS.

Interpretation: VNS successfully reached the global optimum ($Z = 551$) with a **CPU time of 2.002828 seconds**. The shaking mechanism proved vital; by perturbing the solution when VND was stuck at 547, VNS managed to explore a better neighborhood and find the optimal configuration.

7 Comparative Analysis of Results

This section presents a consolidated comparison of all implemented methods. The analysis focuses on the trade-off between solution quality (Utility Z) and computational effort (CPU Time).

7.1 Performance Summary

Before diving into the numerical comparison, the following terminal output illustrates the successful execution and consistency of the metaheuristic approaches across the tested instances.

7.2 Discussion of Findings

- **Heuristic Performance:** Sorting by the profit/weight ratio remains the most reliable deterministic greedy approach, achieving optimal or near-optimal results instantly.

```

PS C:\Users\pc\Desktop\knapsac> python .\table.py

===== RESULTS FOR INSTANCE 1 (n=100) =====
Method          | Utility (Z) | Time (s)
-----
Greedy Ratio     | 551         | 0.000044
Greedy Utility   | 551         | 0.000021
VND              | 551         | 0.000732
VNS              | 551         | 1.500026
Greedy Weight    | 532         | 0.000019

===== RESULTS FOR INSTANCE 2 (n=2000) =====
Method          | Utility (Z) | Time (s)
-----
Greedy Ratio     | 11102       | 0.000705
Greedy Utility   | 11102       | 0.000407
Greedy Weight    | 11102       | 0.000397
VND              | 11102       | 0.309996
VNS              | 11102       | 1.500248
PS C:\Users\pc\Desktop\knapsac> 

```

Figure 2: Combined execution results for VND and VNS.

Table 1: Comprehensive performance comparison across all methods.

Instance (n, W)	Exact	Greedy Heuristics (Approximate)					Metaheuristics	
	CPLEX	Ratio	Utility	Weight	Tie-B.	Rand.	VND	VNS
Instance 1 (100, 1400)	$Z = 551$ (0.040s)	$Z = 551$ (0.00004s)	$Z = 551$ (0.00002s)	$Z = 532$ (0.00002s)	$Z = 551$ (0.00005s)	$Z = 550$ (0.00006s)	$Z = 547$ (0.0006s)	$Z = 551$ (2.0028s)
Instance 2 (2000, 33000)	N/A* (N/A)	$Z = 11102$ (0.0007s)	$Z = 11102$ (0.0004s)	$Z = 11102$ (0.0004s)	$Z = 11102$ (0.0008s)	$Z = 11095$ (0.0009s)	$Z = 11102$ (0.3099s)	$Z = 11102$ (2.0030s)
Winner	Accuracy	Efficiency (Speed)					Local Opt. SE	Global Opt.

- **Metaheuristic Value:** The transition from VND to VNS is justified by the latter’s ability to escape local optima. While VND was faster, VNS ensured the global optimum was reached even with poor starting points.

8 Generalization to the Multidimensional Knapsack Problem (MKP)

The approaches developed for the 0-1 Knapsack problem can be generalized to the Multidimensional Knapsack Problem (MKP), where multiple resource constraints must be satisfied.

8.1 Mathematical Formulation

The objective is to maximize $Z = \sum_{i=1}^n u_i x_i$, subject to m linear constraints:

$$\sum_{i=1}^n w_{ij} x_i \leq W_j, \quad \forall j \in \{1, \dots, m\}$$

8.2 Algorithm Adaptations

1. **Greedy Logic:** The ratio is adapted to a "surrogate" utility based on the sum of normalized weights across all dimensions.
2. **Implementation:** The VND and VNS structures remain the same, but the feasibility check must validate all m constraints for every move.

A Python Source Code

The complete source code for all algorithms is provided in the digital files: `Heuristics.py`, `VND.py`, and `VNS.py`.

A.1 Final Discussion

The experimental results yield several key insights into the behavior of these optimization algorithms:

- **Heuristic Efficiency:** For the provided instances, the Greedy Ratio and Utility variations were exceptionally efficient, matching the optimal utility in most cases. This suggests the instances may have a favorable structure for density-based sorting.
- **Metaheuristic Robustness:** While Greedy methods are fast, they are prone to failure on more complex datasets. VND and VNS provide a safety net; specifically, VNS's stochastic shaking ensures that the search does not stagnate in local optima, as evidenced by its consistent performance.
- **Solver Limitations:** The inability of CPLEX to process Instance 2 reinforces the necessity of metaheuristics for large-scale engineering problems at INPT, where exact solutions become computationally prohibitive.